

Walkthrough script

RelayPay Support Agent · Week 6 · Emmanuel Aboyeji · target length 5:00

Three notes carried over from last week's feedback, and how this script answers them: it **opens on a framing slide** instead of a screen share; it spends a full minute on **edge cases, trade-offs and limitations**; and the **live run ends on a real result** — a ticket and an escalation reference you can read on screen.

Have ready before recording: Slides 1–2, the call page, the admin console, a terminal at the repo root, and the Supabase table view.

0:00 — 0:40 · Slide 1: the business problem

"RelayPay is a B2B payments platform. Their support team answers the same three questions all day — where is my payout, what are the fees, why is my account under review. Every one of those answers already exists: in a knowledge base, or one row in a Postgres database. So a customer waits on hold for a lookup that takes two seconds. That is the problem — not that support is hard, but that the cheap questions are crowding out the expensive ones. What I built is a voice agent that answers the cheap questions from RelayPay's own data, and hands the expensive ones to a human with the context already attached."

Slide 1 shows: the three questions, and one line — **"the answer already exists; the caller is waiting for someone to look it up."**

0:40 — 1:10 · Slide 2: how it is wired

"Vapi handles the audio. It calls one endpoint of mine as an OpenAI-compatible custom LLM. Behind that endpoint, the Claude Agent SDK talks to a custom MCP server over stdio, and the MCP server is the only thing that touches Supabase. The important part of that diagram is what Vapi cannot see: the tools, the database, the knowledge base. The voice vendor handles speech. The agent handles support."

Slide 2 shows: `caller → Vapi → /api/vapi → Agent SDK → MCP server → Supabase`, with the seven tool names underneath.

1:10 — 2:25 · Live call

Start a call on the deployed site.

"I'll ask something it has to look up."

"What's the status of transaction TXN-9001?"

"That came back from the database, not the model's imagination — a payout of 2,400 US dollars, still processing, estimated arrival the 19th of August. And notice what it added: that date has passed. I didn't ask it to check that."

"Can you raise a ticket? I'm Dayo Ade, dayo@example.com."

"Ticket reference read back on the call. That's a real row."

Switch to the admin console — show the conversation, the transcript, and the ticket with its draft email.

"Every turn is here, and so is the ticket. The email is drafted but **not sent**. The agent has no ability to send mail — a human edits and approves it. That is a deliberate limit, not a missing feature."

2:25 — 3:10 · Evidence

Terminal: `node scripts/with-env.mjs node scripts/acceptance.mjs https://...`

"Nine acceptance tests against the deployed service — the same endpoint Vapi calls. Knowledge grounding, clarifying questions, lookups, ticket creation, escalation, refusals, voice transport, and persistence."

Show the final line: **9/9 passed**, then the Supabase counts: 16 turns, 9 tool calls, `TICKET-3061627C`, `ESC-E31B9D5B`.

"It took four runs to get here, and the failures were the useful part. I'll come back to one of them."

3:10 — 4:10 · Edge cases, trade-offs, limitations

"Edge cases. The one that matters most on a voice line: a caller reads out a card number. No prompt can stop that — by the time there's text, the value is already captured. So redaction runs at the earliest possible point, before the model, the tools or any log line sees it. Luhn-checked card numbers, spoken digit runs, CVVs and one-time codes. Forty-six tests. The most interesting failure was the agent **inventing an identifier**. It raised a real ticket and filed it against a conversation that didn't exist, because the MCP server read the conversation id out of the tool arguments — so whatever the model wrote became the foreign key. The audit trail looked fine while being wrong, which is the worst kind of wrong. The id now comes from the server. **Trade-off: latency against model quality.** I run Haiku, not Sonnet. On a warmed session Haiku averages 9.2 seconds a turn against Sonnet's 14.6, with the same answers — because the hard reasoning isn't in the model. Retrieval, escalation rules and verdict logic are in the MCP server. The model routes and speaks. I only made that switch after four adversarial probes — prompt extraction, tool disclosure, balance invention, authority bypass — and Haiku refused all four. **Limitations, honestly.** The first turn of a cold instance is slow, because the Agent SDK spawns a subprocess per call; I hide most of it by warming during the greeting, but not all of it. This cannot run on Vercel at all — the SDK's native CLI is 238 megabytes against a 250 megabyte function limit, which is why it's on Render. Without an embedding key, retrieval is Postgres full-text rather than vector search. And I haven't load-tested concurrency: four simultaneous sessions was enough to make a small instance return 502s."

4:10 — 4:45 · The thing I would change

"If I started again, I'd treat the deployment target as a design input, not a packaging step. I built for Vercel by reflex, and it cannot host this — the build succeeds, then every call fails with an error pointing nowhere. Both blockers were discoverable in an afternoon, before writing a line of code. The second lesson is smaller and I'll keep it: I added a diagnostic header that returns why a turn degraded, visible only to an authenticated caller. The moment it existed it told me the real error instantly. Before that I was guessing at a generic apology."

4:45 — 5:00 · Close

"So: a voice agent that answers from real data, refuses what it can't ground, drafts but never sends, and leaves an audit trail a human can act on. Nine of nine on the deployed service. Repo and live link are in the one-pager."

RECORDING NOTES

- **Do not narrate the architecture over a code editor.** Slides 1 and 2 carry it; the screen share starts at 1:10.
- Let the call play in real time. A ~10 second pause before the first answer is honest, and the follow-up turn is visibly faster — which is the latency story told rather than claimed.
- Have the acceptance run **already finished** in a scrollback. Don't wait on it live; it takes several minutes.
- Say "I don't know" where true. The concurrency limit is unmeasured, and saying so is stronger than implying otherwise.